

INTERASCIE WEB

# Student Dashboard

React Component Code Documentation

A guided reference for `Students.jsx`, `StudentPopup.jsx`, and `StudentCard.jsx` - including their Supabase operations, shared state, props, user interactions, and component connections.

Source snapshot: July 26, 2026

Prepared from the current files in the InteraScie Web workspace.

# 1. System Overview

The Student Dashboard uses a parent-and-child component structure. `Students.jsx` owns the data and database operations. It creates one `StudentCard` for every student and opens `StudentPopup` when the teacher adds or edits a record.

| Part                    | Main responsibility                                                         | Connected to                                           |
|-------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Students.jsx</b>     | Loads, formats, displays, adds, updates, selects, and deletes student data. | Supabase, TeacherPage, StudentCard, StudentPopup       |
| <b>StudentPopup.jsx</b> | Collects student details in a controlled form for adding or editing.        | Students.jsx through onSave and onClose props          |
| <b>StudentCard.jsx</b>  | Displays one student and reports edit/selection actions to its parent.      | Students.jsx through onEdit and onSelect props         |
| <b>Supabase</b>         | Stores Student and Section rows and enforces Row Level Security.            | loadStudents, loadSections, save, and delete functions |

## Main data and event flow

| Action                 | Flow                                                                                       |
|------------------------|--------------------------------------------------------------------------------------------|
| <b>Page opens</b>      | Students.jsx -> Supabase Student/Section tables -> React state -> StudentCard list         |
| <b>Add student</b>     | Add button -> openAddPopup -> StudentPopup -> onSave -> Supabase insert -> reload list     |
| <b>Edit student</b>    | StudentCard -> onEdit(student) -> StudentPopup -> onSave -> Supabase update -> reload list |
| <b>Delete students</b> | Checkboxes -> selectedStudentIDs -> confirmation -> Supabase delete -> reload list         |

Important design idea: the parent owns shared state. The popup and cards do not query Supabase directly; they send user actions upward through callback props. This keeps database logic in one component and makes the child components easier to reuse.

## 2. Students.jsx

src/Components/Students.jsx

This is the main page component. It owns the student list, section list, loading and error states, popup state, selected checkboxes, and every Supabase operation used on this page.

### Imports and dependencies (lines 1-10)

```
1 | import "../css/Students.css";
2 | import StudentCard from "../StudentCard";
3 | import StudentPopup from "../StudentPopup";
4 | import TeacherPage from "../TeacherPage";
5 | import {
6 |   useCallbakc,
7 |   useEffect,
8 |   useState,
9 | } from "react";
10 | import { supabase } from "../lib/supabase";
```

- Line 1 connects the page to its explicit component-specific stylesheet, Students.css.
- Lines 2-4 import the two child components and the shared TeacherPage layout.
- Lines 5-9 import React hooks: useState stores changing values, useEffect runs page-start work, and useCallbakc keeps loader function references stable.
- Line 10 imports the configured Supabase client used for all database requests.

### Component state (lines 12-24)

```
12 | function Students() {
13 |   const [students, setStudents] = useState([]);
14 |   const [sections, setSections] = useState([]);
15 |
16 |   const [loadingStudents, setLoadingStudents] =useState(true);
17 |
18 |   const [studentError, setStudentError] =useState("");
19 |
20 |   const [selectedStudent, setSelectedStudent] =useState(null);
21 |
22 |   const [isPopupOpen, setIsPopupOpen] =useState(false);
23 |
24 |   const [selectedStudentIDs, setSelectedStudentIDs,] = useState([]);
```

- students and sections hold the rows returned by Supabase.
- loadingStudents controls the loading message, while studentError stores a readable error message.
- selectedStudent determines add mode (null) or edit mode (a student object).
- isPopupOpen controls whether StudentPopup is mounted and visible.
- selectedStudentIDs is an array of checked student primary keys and is the source of truth for Select All and bulk deletion.

## Loading and formatting students (lines 26-66)

```
26 | const loadStudents = useCallback(async () => {
27 |   setLoadingStudents(true);
28 |   setStudentError("");
29 |
30 |   const { data, error } = await supabase
31 |     .from("Student")
32 |     .select(`
33 |       studentID,
34 |       firstName,
35 |       lastName,
36 |       username,
37 |       sectionID,
38 |       Section (
39 |         sectionName
40 |       )
41 |     `)
42 |     .order("lastName", { ascending: true });
43 |
44 |   if (error) {
45 |     console.error(error.message);
46 |     setStudentError("Unable to load students.");
47 |     setLoadingStudents(false);
48 |     return;
49 |   }
50 |
51 |   const formattedStudents = data.map(
52 |     (student) => ({
53 |       studentID: student.studentID,
54 |       firstName: student.firstName,
55 |       lastName: student.lastName,
56 |       username: student.username,
57 |       sectionID: student.sectionID,
58 |       section:
59 |         student.Section?.sectionName ??
60 |         "No section",
61 |     })
62 |   );
63 |
64 |   setStudents(formattedStudents);
65 |   setLoadingStudents(false);
66 | }, []);
```

- The async loader first marks the page as loading and clears an earlier error.
- The Supabase query reads Student fields plus the related Section.sectionName through the foreign-key relationship, then sorts by lastName.
- If Supabase returns an error, the function logs the technical message, shows a safe page message, stops loading, and exits.
- data.map creates the simpler object shape expected by StudentCard. Optional chaining and ?? use 'No section' when no related section name exists.
- The final two state updates display the formatted records and remove the loading state.

### Loading sections (lines 68-81)

```
68 |   const loadSections = useCallback(async () => {
69 |     const { data, error } = await supabase
70 |       .from("Section")
71 |       .select("sectionID, sectionName")
72 |       .order("sectionName", { ascending: true });
73 |
74 |     if (error) {
75 |       console.error(error.message);
76 |       setStudentError("Unable to load sections.");
77 |       return;
78 |     }
79 |
80 |     setSections(data);
81 |   }, []);
```

- This query reads sectionID and sectionName so StudentPopup can build its dropdown.
- The rows are alphabetized by sectionName and stored in sections.
- This request is separate from loadStudents because the dropdown needs all allowed sections, not only each student's joined section name.

### Initial page loading effect (lines 83-90)

```
83 |   useEffect(() => {
84 |     const loadTimer = window.setTimeout(() => {
85 |       loadStudents();
86 |       loadSections();
87 |     }, 0);
88 |
89 |     return () => window.clearTimeout(loadTimer);
90 |   }, [loadStudents, loadSections]);
```

- useEffect schedules both loaders when this component mounts.
- The zero-delay timer moves the state-changing loaders into a callback and satisfies the project's React effect lint rule.
- The cleanup cancels the timer if the component unmounts before it runs.
- The dependency array reruns the effect only if either memoized loader reference changes.

### Opening and closing the popup (lines 92-105)

```
92 |   function openAddPopup() {
93 |     setSelectedStudent(null);
94 |     setIsPopupOpen(true);
95 |   }
96 |
97 |   function openEditPopup(student) {
98 |     setSelectedStudent(student);
99 |     setIsPopupOpen(true);
100 |   }
101 |
102 |   function closePopup() {
103 |     setSelectedStudent(null);
104 |     setIsPopupOpen(false);
105 |   }
```

- openAddPopup clears selectedStudent, so the popup knows it is creating a new record.
- openEditPopup stores the clicked student, allowing the popup fields to begin with that student's values.
- closePopup clears the selected record and removes the popup from the page.

## Saving an added or edited student (lines 107-141)

```
107 | async function handleSaveStudent(studentData) {  
108 |     const payload = {  
109 |         firstName: studentData.firstName.trim(),  
110 |         lastName: studentData.lastName.trim(),  
111 |         username: studentData.username.trim(),  
112 |         sectionID: studentData.sectionID,  
113 |     };  
114 |  
115 |     let saveResult;  
116 |  
117 |     if (selectedStudent) {  
118 |         saveResult = await supabase  
119 |             .from("Student")  
120 |             .update(payload)  
121 |             .eq(  
122 |                 "studentID",  
123 |                 selectedStudent.studentID  
124 |             )  
125 |             .select("studentID")  
126 |             .single();  
127 |     } else {  
128 |         saveResult = await supabase  
129 |             .from("Student")  
130 |             .insert(payload)  
131 |             .select("studentID")  
132 |             .single();  
133 |     }  
134 |  
135 |     if (saveResult.error) {  
136 |         throw saveResult.error;  
137 |     }  
138 |  
139 |     await loadStudents();  
140 |     closePopup();  
141 | }
```

- The payload trims text fields before sending them to Supabase and carries the selected sectionID.
- When selectedStudent exists, update changes the Student row whose studentID matches the selected card.
- When selectedStudent is null, insert creates a new Student row.
- select(...).single() asks Supabase to return the affected row as one object, which makes an RLS failure or database error observable.
- Errors are thrown back to StudentPopup, where they are displayed. A successful save reloads the cards and closes the popup.

**Individual and Select All checkbox state (lines 143-169)**

```
143 | function handleStudentSelection(studentID, isChecked) {  
144 |   if (isChecked) {  
145 |     setSelectedStudentIDs((currentIDs) =>  
146 |       currentIDs.includes(studentID)  
147 |         ? currentIDs  
148 |         : [...currentIDs, studentID]  
149 |     );  
150 |     return;  
151 |   }  
152 |  
153 |   setSelectedStudentIDs((currentIDs) =>  
154 |     currentIDs.filter(  
155 |       (currentID) => currentID !== studentID  
156 |     )  
157 |   );  
158 | }  
159 |  
160 | function handleSelectAll(event) {  
161 |   if (event.target.checked) {  
162 |     setSelectedStudentIDs(  
163 |       students.map((student) => student.studentID)  
164 |     );  
165 |     return;  
166 |   }  
167 |  
168 |   setSelectedStudentIDs([]);  
169 | }
```

- handleStudentSelection adds a checked ID only if it is not already present, preventing duplicates.
- Unchecking uses filter to create a new array without that studentID.
- handleSelectAll replaces the array with every loaded studentID when checked and with an empty array when unchecked.
- Functional state updates use currentIDs so rapid checkbox clicks always work with the latest state.

**Confirmed bulk deletion (lines 171-201)**

```
171 | async function handleRemoveSelected() {
172 |   if (selectedStudentIDs.length === 0) {
173 |     alert("Please select at least one student.");
174 |     return;
175 |   }
176 |
177 |   const confirmed = window.confirm(
178 |     "Are you sure you want to delete the selected students?"
179 |   );
180 |
181 |   if (!confirmed) {
182 |     return;
183 |   }
184 |
185 |   const { data: deletedStudents, error } =
186 |     await supabase
187 |       .from("Student")
188 |       .delete()
189 |       .in("studentID", selectedStudentIDs)
190 |       .select("studentID");
191 |
192 |   if (error) {
193 |     console.error(error.message);
194 |     alert("Unable to delete the selected students.");
195 |     return;
196 |   }
197 |
198 |   setSelectedStudentIDs([]);
199 |   await loadStudents();
200 |   alert(`${deletedStudents.length} student(s) deleted.`);
201 | }
```

- The first guard prevents an empty delete request.
- `window.confirm` displays the browser's OK/Cancel question. Cancel immediately exits the function.
- `delete().in(...)` deletes rows whose `studentID` appears in `selectedStudentIDs`. `select('studentID')` returns the successfully deleted IDs.
- On success, the selection is cleared, the student list is reloaded, and the deleted count is shown.
- Supabase still decides whether deletion is allowed through RLS policies and foreign-key constraints.

## Page layout and toolbar (lines 203-229)

```
203 |   return (  
204 |     <TeacherPage title="Students">  
205 |       <section className="studentspanel">  
206 |         <div className="studentbox">  
207 |           <div className="tools">  
208 |             <div className="searchbar">  
209 |               <input  
210 |                 type="text"  
211 |                 placeholder="Search students..."  
212 |               />  
213 |             </div>  
214 |  
215 |             <div className="buttons">  
216 |               <button type="button" id="batch">  
217 |                 Batch Upload  
218 |               </button>  
219 |  
220 |               <button  
221 |                 type="button"  
222 |                 id="remove"  
223 |                 onClick={handleRemoveSelected}  
224 |                 disabled={selectedStudentIDs.length === 0}  
225 |               >  
226 |                 Remove Selected  
227 |               </button>  
228 |             </div>  
229 |           </div>  
230 |         </div>  
231 |       </section>  
232 |     </TeacherPage>  
233 |   )  
234 | }
```

- TeacherPage supplies the reusable teacher layout and receives Students as the page title.
- The search input and Batch Upload button are currently visual controls only because neither has an event handler yet.
- Remove Selected calls handleRemoveSelected and stays disabled while selectedStudentIDs is empty.

**Rendering cards and Select All (lines 231-269)**

```

231 |         <div className="studentcontainer">
232 |             {loadingStudents && (
233 |                 <p>Loading students...</p>
234 |             )}
235 |
236 |             {studentError && (
237 |                 <p>{studentError}</p>
238 |             )}
239 |
240 |             {!loadingStudents &&
241 |             !studentError &&
242 |             students.map((student) => (
243 |                 <StudentCard
244 |                     key={student.studentID}
245 |                     student={student}
246 |                     onEdit={openEditPopup}
247 |                     isSelected={selectedStudentIDs.includes(
248 |                         student.studentID
249 |                     )}
250 |                     onSelect={handleStudentSelection}
251 |                 />
252 |             ))}
253 |         </div>
254 |
255 |         <div className="check">
256 |             <input
257 |                 type="checkbox"
258 |                 id="select-all"
259 |                 checked={
260 |                     students.length > 0 &&
261 |                     selectedStudentIDs.length === students.length
262 |                 }
263 |                 onChange={handleSelectAll}
264 |             />
265 |
266 |             <label htmlFor="select-all">
267 |                 Select All
268 |             </label>
269 |         </div>

```

- Conditional JSX displays loading or error text before displaying the list.
- `students.map` creates one `StudentCard` per object. `key` uses the database primary key so React can track each card reliably.
- `student` and callback props connect each card to the parent. `isSelected` makes every checkbox controlled by parent state.
- The Select All checkbox is checked only when at least one student exists and every loaded student ID is selected.

**Add button, popup connection, and export (lines 270-294)**

```
270 |         </div>
271 |
272 |         <div className="add">
273 |             <button
274 |                 type="button"
275 |                 onClick={openAddPopup}
276 |             >
277 |                 +
278 |             </button>
279 |         </div>
280 |     </section>
281 |
282 |     {isPopupOpen && (
283 |         <StudentPopup
284 |             student={selectedStudent}
285 |             sections={sections}
286 |             onClose={closePopup}
287 |             onSave={handleSaveStudent}
288 |         />
289 |     )}
290 | </TeacherPage>
291 | );
292 | }
293 |
294 | export default Students;
```

- The plus button calls openAddPopup, which opens an empty form.
- StudentPopup is conditionally created only while isPopupOpen is true.
- The parent passes the selected student, all section options, and its close/save functions as props.
- export default allows the router or another component to import and display Students.

## 3. StudentPopup.jsx

src/Components/StudentPopup.jsx

This child component is a reusable modal form. The same form supports both adding and editing because the student prop may contain an existing record or may be null.

### Imports, props, and mode (lines 1-11)

```
1 | import "../css/StudentPopup.css";
2 | import { useState } from "react";
3 | import studentImage from "../assets/pfp.png";
4 |
5 | function StudentPopup({
6 |   student,
7 |   sections,
8 |   onClose,
9 |   onSave,
10 | }) {
11 |   const isEditing = Boolean(student);
```

- The component imports only its own CSS, React state, and the student placeholder image.
- student supplies existing values, sections supplies dropdown choices, and onClose/onSave are functions owned by Students.jsx.
- Boolean(student) becomes true for edit mode and false for add mode.

### Controlled form state (lines 13-32)

```
13 |   const [firstName, setFirstName] = useState(
14 |     student?.firstName ?? ""
15 |   );
16 |
17 |   const [lastName, setLastName] = useState(
18 |     student?.lastName ?? ""
19 |   );
20 |
21 |   const [username, setUsername] = useState(
22 |     student?.username ?? ""
23 |   );
24 |
25 |   const [sectionID, setSectionID] = useState(
26 |     student?.sectionID
27 |     ? String(student.sectionID)
28 |     : ""
29 |   );
30 |
31 |   const [saving, setSaving] = useState(false);
32 |   const [saveError, setSaveError] = useState("");
```

- Each field has its own state value and setter. Existing student values prefill edit mode; ?? uses an empty string in add mode.
- sectionID is converted to a string because HTML select values are strings.
- saving prevents repeated submissions, while saveError holds a message returned from the parent's save operation.

**Form submission (lines 34-52)**

```
34 |   async function handleSubmit(event) {  
35 |     event.preventDefault();  
36 |  
37 |     setSaving(true);  
38 |     setSaveError("");  
39 |  
40 |     try {  
41 |       await onSave({  
42 |         firstName,  
43 |         lastName,  
44 |         username,  
45 |         sectionID: Number(sectionID),  
46 |       });  
47 |     } catch (error) {  
48 |       console.error(error.message);  
49 |       setSaveError(error.message);  
50 |       setSaving(false);  
51 |     }  
52 |   }
```

- `preventDefault` stops the browser from refreshing the page when the form submits.
- The form enters a saving state and clears an older error before calling `onSave`.
- The popup packages its current fields into one object. `Number(sectionID)` converts the dropdown value for a numeric database column.
- If `Students.jsx` throws a Supabase error, `catch` displays it and re-enables the Save button.

## Overlay, popup shell, and close behavior (lines 54-77)

```

54 |   return (
55 |     <div
56 |       className="studentpopupoverlay"
57 |       onMouseDown={onClose}
58 |     >
59 |       <div
60 |         className="studentpopup"
61 |         onMouseDown={(event) =>
62 |           event.stopPropagation()
63 |         }
64 |       >
65 |         <div className="studentpopupheader">
66 |           <h2>
67 |             {isEditing ? "Edit Student" : "Add Student"}
68 |           </h2>
69 |
70 |           <button
71 |             type="button"
72 |             className="closestudentpopup"
73 |             onClick={onClose}
74 |           >
75 |             ×
76 |           </button>
77 |         </div>

```

- Clicking or pressing the mouse on the overlay calls onClose.
- stopPropagation on the inner popup prevents a click inside the form from reaching the overlay and closing it.
- The heading changes between Edit Student and Add Student using isEditing.
- The close button calls the same onClose callback as the overlay.

## Form connection and first-name field (lines 79-102)

```

79 |     <form
80 |       className="studentpopupcontent"
81 |       onSubmit={handleSubmit}
82 |     >
83 |       <img
84 |         className="studentpopupimage"
85 |         src={studentImage}
86 |         alt="Student"
87 |       />
88 |
89 |       <div className="studentpopupform">
90 |         <label htmlFor="student-first-name">
91 |           First name
92 |         </label>
93 |
94 |         <input
95 |           type="text"
96 |           id="student-first-name"
97 |           value={firstName}
98 |           onChange={(event) =>
99 |             setFirstName(event.target.value)
100 |           }
101 |           required
102 |         />

```

- onSubmit connects the form to handleSubmit, so pressing Enter or clicking Save uses the same logic.
- The image is decorative profile content styled through StudentPopup.css.
- The first-name input is controlled: value comes from firstName state and onChange stores every edit.
- htmlFor and id connect the label to its input; required enables browser validation.

**Last-name and username fields (lines 104-130)**

```

104 |         <label htmlFor="student-last-name">
105 |             Last name
106 |         </label>
107 |
108 |         <input
109 |             type="text"
110 |             id="student-last-name"
111 |             value={lastName}
112 |             onChange={(event) =>
113 |                 setLastName(event.target.value)
114 |             }
115 |             required
116 |         />
117 |
118 |         <label htmlFor="student-username">
119 |             Username
120 |         </label>
121 |
122 |         <input
123 |             type="text"
124 |             id="student-username"
125 |             value={username}
126 |             onChange={(event) =>
127 |                 setUsername(event.target.value)
128 |             }
129 |             required
130 |         />

```

- Both inputs follow the same controlled-input pattern as first name.
- React state is the single source of truth, which allows handleSubmit to read all form values without querying the DOM.
- required prevents submission while either value is blank, although database rules remain the final validation layer.

**Section dropdown (lines 132-157)**

```

132 |         <label htmlFor="student-section">
133 |             Section
134 |         </label>
135 |
136 |         <select
137 |             id="student-section"
138 |             value={sectionID}
139 |             onChange={(event) =>
140 |                 setSectionID(event.target.value)
141 |             }
142 |             required
143 |         >
144 |             <option value="" disabled>
145 |                 Select Section
146 |             </option>
147 |
148 |             {sections.map((section) => (
149 |                 <option
150 |                     key={section.sectionID}
151 |                     value={section.sectionID}
152 |                 >
153 |                     {section.sectionName}
154 |                 </option>
155 |             ))}
156 |         </select>
157 |     </div>

```

- value and onChange make the select a controlled field tied to sectionID state.
- The disabled empty option is the initial prompt; required means the teacher must choose a section before saving.
- sections.map creates one option from every Section row loaded by Students.jsx.
- The option key helps React track options, while value is the ID saved to Student.sectionID and visible text is sectionName.

**Errors, actions, and export (lines 159-188)**

```
159 |         {saveError && (  
160 |             <p className="studentpopuperror">  
161 |                 {saveError}  
162 |             </p>  
163 |         )}  
164 |  
165 |         <div className="studentpopupactions">  
166 |             <button  
167 |                 type="button"  
168 |                 className="cancelstudent"  
169 |                 onClick={onClose}  
170 |             >  
171 |                 Cancel  
172 |             </button>  
173 |  
174 |             <button  
175 |                 type="submit"  
176 |                 className="savestudent"  
177 |                 disabled={saving}  
178 |             >  
179 |                 {saving ? "Saving..." : "Save"}  
180 |             </button>  
181 |         </div>  
182 |     </form>  
183 | </div>  
184 | </div>  
185 | );  
186 | }  
187 |  
188 | export default StudentPopup;
```

- The error paragraph appears only when saveError contains a message.
- Cancel closes the popup without submitting because its type is button.
- Save submits the form. While saving is true, it is disabled and its label changes to Saving... to prevent duplicate requests.
- The component is exported so Students.jsx can import and render it.

## 4. StudentCard.jsx

src/Components/StudentCard.jsx

This is a presentational child component for one student. It does not own the database or the selection list; it displays props and reports interactions back to Students.jsx.

### Imports and component contract (lines 1-9)

```
1 | import "../css/StudentCard.css";
2 | import studentImage from "../assets/pfp.png";
3 |
4 | function StudentCard({
5 |   student,
6 |   onEdit,
7 |   isSelected,
8 |   onSelect,
9 | }) {
```

- The card imports its own CSS and the shared profile image.
- student contains displayed data, onEdit handles card clicks, isSelected controls the checkbox, and onSelect reports checkbox changes.
- These props form the contract between StudentCard and its parent Students.jsx.

### Clickable card and displayed information (lines 10-24)

```
10 |   return (
11 |     <article className="studentcard" onClick={() => onEdit(student)}>
12 |       <img
13 |         className="studentcardimage"
14 |         src={studentImage}
15 |         alt={` ${student.firstName} ${student.lastName}`}
16 |       />
17 |
18 |       <div className="studentinformation">
19 |         <h3>
20 |           {student.lastName}, {student.firstName}
21 |         </h3>
22 |         <p>Username: {student.username}</p>
23 |         <p>Section: {student.section}</p>
24 |       </div>
```

- Clicking the article calls onEdit(student), sending the complete student object back to Students.jsx.
- The parent then stores that object and opens StudentPopup in edit mode.
- The image alt text is generated from the student's name for accessibility.
- The text area displays last name, first name, username, and the formatted section name.

## Controlled selection checkbox (lines 26-43)

```
26 |     <input
27 |       className="studentcheckbox"
28 |       type="checkbox"
29 |       checked={isSelected}
30 |       aria-label={`Select ${student.firstName} ${student.lastName}`}
31 |       onChange={(event) =>
32 |         onSelect(
33 |           student.studentID,
34 |           event.target.checked
35 |         )
36 |       }
37 |       onClick={(event) => event.stopPropagation()}
38 |     />
39 |   </article>
40 | );
41 | }
42 |
43 | export default StudentCard;
```

- checked={isSelected} means the checkbox display always follows selectedStudentIDs in Students.jsx.
- onChange sends the studentID and new checked value through onSelect, letting the parent update its array.
- stopPropagation prevents a checkbox click from also triggering the card's edit-popup click handler.
- The aria-label gives screen-reader users a student-specific name for the checkbox.

## 5. How the Components Connect

### Props passed from Students.jsx

| Destination         | Prop       | Meaning                                                      |
|---------------------|------------|--------------------------------------------------------------|
| <b>StudentCard</b>  | student    | The formatted student object to display and later edit.      |
| <b>StudentCard</b>  | onEdit     | Reference to openEditPopup; called when the card is clicked. |
| <b>StudentCard</b>  | isSelected | Boolean calculated from selectedStudentIDs.                  |
| <b>StudentCard</b>  | onSelect   | Reference to handleStudentSelection; called by the checkbox. |
| <b>StudentPopup</b> | student    | Selected record for edit mode, or null for add mode.         |
| <b>StudentPopup</b> | sections   | Section rows used to generate dropdown options.              |
| <b>StudentPopup</b> | onClose    | Reference to closePopup.                                     |
| <b>StudentPopup</b> | onSave     | Reference to handleSaveStudent, which writes to Supabase.    |

### Supabase connections

- Student is read by loadStudents, inserted or updated by handleSaveStudent, and deleted by handleRemoveSelected.
- Section is read by loadSections and is also joined inside loadStudents to obtain sectionName.
- Student.sectionID is the value selected in StudentPopup and should reference Section.sectionID.
- The signed-in teacher's Row Level Security policies decide which Student and Section rows these frontend requests may access.
- Foreign-key delete rules decide whether related records, such as StudentAssessment rows, allow or block deletion.

## Current implementation notes

| Observation                          | What it means                                                                                                                                                     |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Search is not connected</b>       | The input is visible but has no search state or filtering handler yet.                                                                                            |
| <b>Batch Upload is not connected</b> | The button currently has no onClick function.                                                                                                                     |
| <b>Section is required</b>           | The current disabled empty option plus required attribute does not allow a null section through this form.                                                        |
| <b>Delete needs RLS</b>              | A valid DELETE policy is required. Related StudentAssessment rows may still block deletion depending on the foreign key rule.                                     |
| <b>ID type assumption</b>            | Number(sectionID) is correct only if sectionID is numeric. It must remain a string if the database uses UUIDs.                                                    |
| <b>Close character</b>               | If the close button displays unusual characters, save StudentPopup.jsx as UTF-8 and replace the glyph with a normal x or a correctly encoded multiplication sign. |

Summary: Students.jsx is the coordinator, StudentCard is the reusable display and selection unit, and StudentPopup is the reusable add/edit form. Data travels downward as props; clicks and submitted form values travel upward as callback calls; only the parent communicates with Supabase.